

1.1 - Basic Computing Concepts

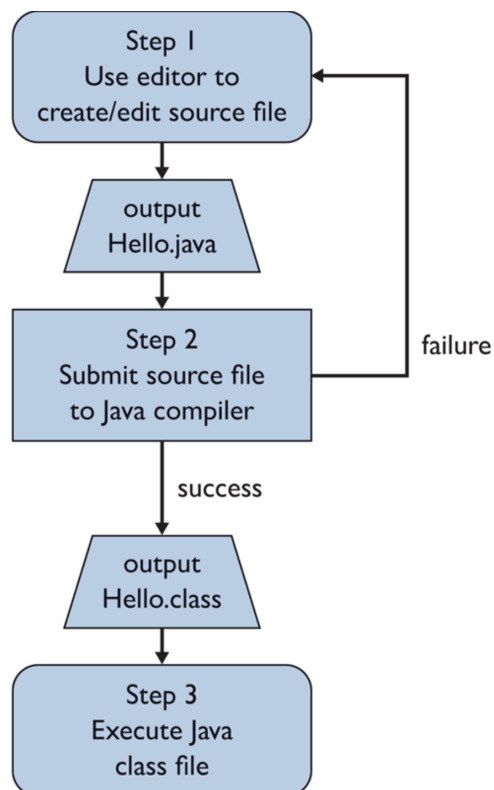
No matter what environment you use you will follow the same three basic steps:

1. Type in a program as a Java class
2. Compile the program file
3. Run the compiled version of the program

The java program files that are used create the extension .java; when compiling a java program the bytecodes are stored in a file with the same name and the extension .class

```
1 public class Hello {  
2     public static void main(String[] args) {  
3         System.out.println("Hello, world!");  
4     }  
5 }
```

Creation and Execution of a Java Program



1.2 - And Now Java

Class

A unit of code that is the basic block building of Java programs

ALL CLASS NAMES SHOULD START WITH A CAPITAL LETTER!!!

“Hello World” program (most basic program)

```
public class Hello {  
    public static void main(String[] args) {  
        system.out.println(“Hello, world!”);  
    }  
}
```

This program defines a class called Hello. Class names always begin with a capital letter, which makes it easier to recognize them. Since Java requires that the class name and the file name match.

Basic Form of a Java class is as it follows:

```
public class <name> {  
    <method>  
    <method>  
    ...  
    <method>  
}
```

The first line known as the class header. The word public in the header indicates that the class is available to anyone to use it.

Methods

Method → A program unit that represents a particular action or computation.

Methods represent a single action or calculation to be performed. A minimum program requires a special method that is known as the main method. It has the following syntax:

```

public static void main(String[] args) {
    <statement>;
    <statement>;
    ...
    <statement>;
}

```

you want to open a Java Static Program?? You say **public static void main(String[] args) {**

Statement

Statement → An executable snippet of code that represents a complete command.

*each statement is terminated by a semicolon (;)

Ex:

```
System.out.println("Hello, world!");
```

Consider the following ex Hello2 which has four lines of code to be executed:

```

public class Hello2 {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
        System.out.println();
        System.out.println("This program produces four");
        System.out.println("lines of output");
    }
}

```

String Literals (Strings)

Strings → Literal text to send to the console window as output.

Common Escape Sequences

Sequence	Represents
\t	tab character
\n	newline character
\"	quotation mark
\\	backslash character

Ex:

```
System.out.println("What \"characters\" does this \\ print?");
```

Result:

What "characters" does this \ print?

Ex:

```
System.out.println("This\nproduces 3 lines\nof output.");
```

Result:

This
produces 3 lines
of output

Print versus Println

Java has a variation of println command called print that allows you to produce output on the current line without going to a new line of output. The println command does two different things. It sends output to the current line and to the beginning of a new line, while the print command only send the output to a current line.

Identifiers and Keywords

The words that are used to name parts of a java command are called identifiers.

Identifier

A name given to an entity in a program, such as a class or method.

Identifiers must be started with a letter, which can be followed by any number of letters or digits.

The following are all legal identifiers:

first, hiThere, numStudents, TwoBy4

The Java language specification defines the set of letters to include the underscore and \$ sign characters **which means these are also legal identifiers:**

two_plus_two, _count, \$2donuts, MAX_COUNT

Illegal identifiers:

two+two, hi there, hi-there, 2by4

For example suppose you were going to put the words “all my children” into an identifier.

AllMyChildren → class name (each word starts with a capital)

allMyChildren → method name (starts with a lower case letter, other words are capitalized)

ALL_MY_CHILDREN → constant name (all uppercase, words separated by underscores)

Java has a set of predefined identifiers called **keywords**, that are reserved for particular uses.

List of Java Keywords:

<code>abstract</code>	<code>continue</code>	<code>for</code>
<code>assert</code>	<code>default</code>	<code>goto</code>
<code>boolean</code>	<code>do</code>	<code>if</code>
<code>break</code>	<code>double</code>	<code>implements</code>
<code>byte</code>	<code>else</code>	<code>import</code>
<code>case</code>	<code>enum</code>	<code>instanceof</code>
<code>catch</code>	<code>extends</code>	<code>int</code>
<code>char</code>	<code>final</code>	<code>interface</code>
<code>class</code>	<code>finally</code>	<code>long</code>
<code>const</code>	<code>float</code>	<code>native</code>
<code>new</code>	<code>switch</code>	
<code>package</code>	<code>synchronized</code>	
<code>private</code>	<code>this</code>	
<code>protected</code>	<code>throw</code>	
<code>public</code>	<code>throws</code>	
<code>return</code>	<code>transient</code>	
<code>short</code>	<code>try</code>	
<code>static</code>	<code>void</code>	
<code>strictfp</code>	<code>volatile</code>	
<code>super</code>	<code>while</code>	

Complex Example:

```
1 public class DrawFigures1 {
2     public static void main(String[] args) {
3         System.out.println("  /\");
4         System.out.println(" /  \");
5         System.out.println("/    \");
6         System.out.println("\    /");
7         System.out.println("\  /");
8         System.out.println("\ /");
9         System.out.println();
10        System.out.println("\    /");
11        System.out.println("\  /");
12        System.out.println("\ /");
13        System.out.println("/\");
14        System.out.println("/  \");
15        System.out.println("/    \");
16        System.out.println();
17        System.out.println("  /\");
18        System.out.println(" /  \");
19        System.out.println("/    \");
20        System.out.println("+-----+");
21        System.out.println("|         |");
22        System.out.println("|         |");
23        System.out.println("+-----+");
24        System.out.println("|United|");
25        System.out.println("|States|");
26        System.out.println("+-----+");
27        System.out.println("|         |");
28        System.out.println("|         |");
29        System.out.println("+-----+");
30        System.out.println("  /\");
31        System.out.println(" /  \");
32        System.out.println("/    \");
33    }
34 }
```

Output:

```
  /\
 /  \
/\    \
\    /
 \  /
  \/
   /\
  /\
 /\
/\
+-----+
|         |
|         |
+-----+
|United|
|States|
+-----+
|         |
|         |
+-----+
  /\
 /  \
/\    \
```

Comments and Readability

Comment → Text that programmers include in a program to explain their code. The compiler ignores comments.

Two ways of commenting

1. `/*` like this `*/`
2. `//` or like this (the comment can only be written like this in a separate line)

1.3 Program Errors

Syntax Errors → Misuse Java (programming equivalent to of bad grammar)

Logic Errors → Code that doesn't perform the task it's intended to perform

Runtime Errors → Logic errors that are so severe that Java stops your program from executing

Syntax Error Examples:

COMMON PROGRAMMING ERROR

File Name Does Not Match Class Name

As mentioned earlier, Java requires that a program's class name and file name match. For example, a program that begins with `public class Hello` must be stored in a file called `Hello.java`.

If you use the wrong file name (for example, saving it as `WrongFileName.java`), you'll get an error message like this:

```
WrongFileName.java:1: error: class Hello is public,  
    should be declared in a file named Hello.java  
public class Hello {  
    ^  
1 error
```

COMMON PROGRAMMING ERROR

Misspelled Words

Java (like most programming languages) is very picky about spelling. You need to spell each word correctly, including proper capitalization. Suppose, for example, that you were to replace the `println` statement in the "hello world" program with the following:

```
System.out.pruntln("Hello, world!");
```

When you try to compile this program, it will generate an error message similar to the following:

```
Hello.java:3: error: cannot find symbol  
symbol : method pruntln(java.lang.String)  
location: variable out of type PrintStream  
    System.out.pruntln("Hello, world!");  
                ^  
1 error
```


COMMON PROGRAMMING ERROR

Forgetting a Semicolon

All Java statements must end with semicolons, but it's easy to forget to put a semicolon at the end of a statement, as in the following program:

```
1 public class MissingSemicolon {
2     public static void main(String[] args) {
3         System.out.println("A rose by any other name")
4         System.out.println("would smell as sweet");
5     }
6 }
```

In this case, the compiler produces output similar to the following:

```
MissingSemicolon.java:3: error: ';' expected
    System.out.println("A rose by any other name")
                                ^
1 error
```

COMMON PROGRAMMING ERROR

Forgetting a Required Keyword

Another common syntax error is to forget a required keyword when you are typing your program, such as `static` or `class`. Double-check your programs against the examples in the textbook to make sure you haven't omitted an important keyword.

The compiler will give different error messages depending on which keyword is missing, but the messages can be hard to understand. For example, you might write a program called `Bug4` and forget the keyword `class` when writing its class header. In this case, the compiler will provide the following error message:

```
Bug4.java:1: error: class, interface, or enum expected
public Bug4 {
    ^
1 error
```

However, if you forget the keyword `void` when declaring the `main` method, the compiler generates a different error message:

```
Bug5.java:2: error: invalid method declaration; return type required
    public static main(String[] args) {
                ^
1 error
```

Logic Errors → Also called bugs. Programmers use words like “bug-ridden” and “buggy” to describe poorly written programs, and the process of finding and eliminating bugs is called “debugging”

1.4 Procedural Decomposition

Decomposition → A separation into discernible parts, each of which is simpler than the whole.

For example:

Dividing a complex text into a set of subtasks.

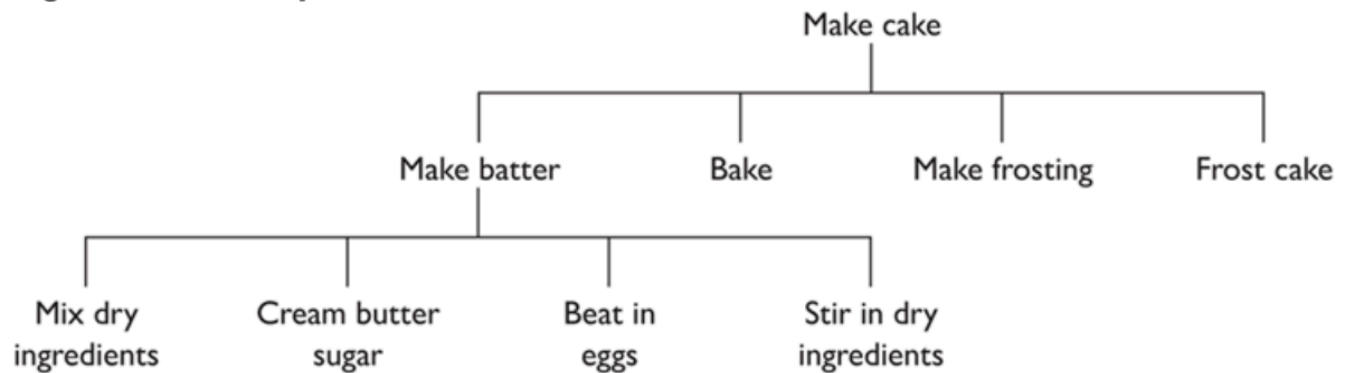
As an example of Procedural Decomposition consider the following:

- Make the cake
- Bake the cake
- Make the frosting
- Frost the cake

Each of these four tasks has details associated with it. To make the batter for example you follow these steps:

- Mix the dry ingredients
- Cream the butter and sugar
- Beat in the eggs
- Stir in the dry ingredients

Figure 1.2 Decomposition of “Make cake” task



Another problem solving term has to do with the process of programming, where programmers instead of trying to complete a working program all at once, they choose some piece of the problem to implement first. Then add another piece, and another, and another...

The overall program is built up slowly piece by piece, and this process is known as iterative enhancement or stepwise refinement.

Iterative Enhancement

The process of producing a program in stages, adding new functionality at each stage. A key feature of each iterative step is that you can test it to make sure the pieces work before moving on.

Static Method

A block of java statements that is given a name

Example:

```
public static void drawBox(){
    System.out.println("+ ____ +");
    System.out.println("|      |");
    System.out.println("|      |");
    System.out.println("+ ____ +");
}
```

public indicates that this method is available for use by all parts of the program.

static indicates that this is a static (procedural-style, not object-oriented) method.

void indicates that this method executes statements but does not produce any value.

drawBox name of the method

Method Call:

A command to execute another method, which causes all of the statements inside that method to be executed

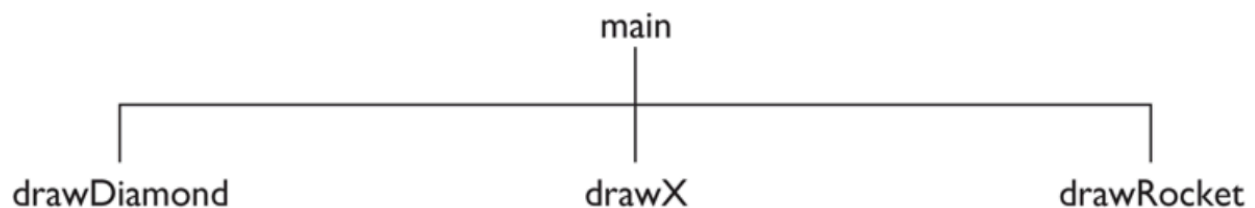
Flow of Control:

The order in which the statements of a Java program are executed

1.5 Case Study DrawFigures

In the first code many println statements were being printed. Instead of using the public static void to run all your lines of code you can divide them into different subfigures.

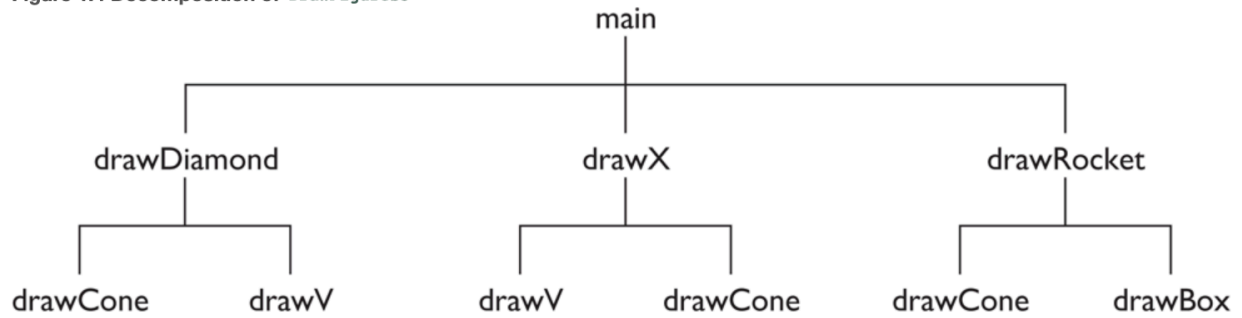
```
1 // This program draws three text figures using
2 // a static method to represent each figure.
3 public class DrawFigures2 {
4     public static void main(String[] args) {
5         drawDiamond();
6         drawX();
7         drawRocket();
8     }
9
10    public static void drawDiamond() {
11        System.out.println("  /\");
12        System.out.println(" /  \");
13        System.out.println("/    \");
14        System.out.println("\  /");
15        System.out.println("\  /");
16        System.out.println("\ /");
17        System.out.println();
18    }
19
20    public static void drawX() {
21        System.out.println("\  /");
22        System.out.println("\  /");
23        System.out.println("\ /");
24        System.out.println("/  \");
25        System.out.println("/  \");
26        System.out.println("/  \");
27        System.out.println();
28    }
29
30    public static void drawRocket() {
31        System.out.println("  /\");
32        System.out.println(" /  \");
33        System.out.println("/    \");
34        System.out.println("++++++");
35        System.out.println("|      |");
36        System.out.println("|      |");
37        System.out.println("++++++");
38        System.out.println("|United|");
39        System.out.println("|States|");
40        System.out.println("++++++");
41        System.out.println("|      |");
42        System.out.println("|      |");
43        System.out.println("++++++");
44        System.out.println("  /\");
45        System.out.println(" /  \");
46        System.out.println("/    \");
47    }
48 }
```



By doing the following you are dividing the arguments into 3 different categories. But the code can be even better and improved like this:

```
1 // This program draws three text figures using static
2 // methods for structure and to avoid redundancy.
3 public class DrawFigures3 {
4     public static void main(String[] args) {
5         drawDiamond();
6         drawX();
7         drawRocket();
8     }
9
10    public static void drawDiamond() {
11        drawCone();
12        drawV();
13        System.out.println();
14    }
15
16    public static void drawX() {
17        drawV();
18        drawCone();
19        System.out.println();
20    }
21
22    public static void drawRocket() {
23        drawCone();
24        drawBox();
25        System.out.println("|United|");
26        System.out.println("|States|");
27        drawBox();
28        drawCone();
29        System.out.println();
30    }
31
32    public static void drawBox() {
33        System.out.println("+-----+");
34        System.out.println("|       |");
35        System.out.println("|       |");
36        System.out.println("+-----+");
37    }
38
39    public static void drawCone() {
40        System.out.println("  /\\"");
41        System.out.println(" /  \\"");
42        System.out.println("/    \\"");
43    }
44
45    public static void drawV() {
46        System.out.println("  \\/");
47        System.out.println("  \\/");
48        System.out.println("  \\/");
49    }
50 }
```

Figure 1.4 Decomposition of `DrawFigures3`



Here is a breakdown on the method:

```
1st  main
2nd   drawDiamond
3rd    drawCone
4th    drawV
5th   drawX
6th    drawV
7th    drawCone
8th   drawRocket
9th    drawCone
10th   drawBox
11th   drawBox
12th   drawCone
```

2.1 Basic Data Concepts

Data Type

A name for a category of data values that are all related, as in type `int` in Java which is used to represent integer values.

Primitive Types

There are eight primitive data types in java: `boolean`, `byte`, `char`, `double`, `float`, `int`, `long`, and `short`.

Four of those are considered **fundamental: `boolean`, `char`, `double` and `int`**. The other four types are variations that exist for programs that have special requirements.

Type	Description	Examples
<code>int</code>	integers (whole numbers)	42, -3, 18, 0
<code>double</code>	real numbers	7.35, 14.9
<code>char</code>	single characters	"a", "x", "t"
<code>boolean</code>	logical values	true, false

When you write programs, you will often need to include values and calculations. The technical term for these elements are expressions

Expression:

A simple value or a set of operations that produces a value.

Evaluation:

The process of obtaining the value of an expression

Operator:

A special symbol (like `+` or `*`) this is used to indicate an operation to be performed on one or more values

Operand:

Numbers being used with special symbols (like `6 + 9`, `20 - 21`, `15 * 30`, etc)

Operator	Meaning	Example	Result
+	addition	$2 + 2$	4
-	subtraction	$2 - 2$	0
*	multiplication	$2 * 2$	4
/	division	$2 / 2$	1
%	remainder or mod	$19 \% 5$	4

Precedence:

The binding power of an operator, which determines how to group parts of an expression.

Ex:

$$2 + 3 * 4 = 2 + 12 = 14$$

2.2 Variables

Variable:

A memory location with a name and a type that stores a value

Example: (BMI Program)

// Computes Body Mass Index (BMI) for a specific person

```
public class BMICalculator {  
    public static void main(String[] args) {  
        // declare variables  
        double height;  
        double weight;  
        double bmi;  
  
        // compute BMI  
        height = 70;  
        weight = 195;  
        bmi = weight / (height * height) * 703;  
  
        // print results  
        System.out.println("Current BMI:");  
        System.out.println(bmi);  
    }  
}
```

Output:

27.97

Declaration:

A request to set aside a new variable with a given name and type.

In our case:

```
double height;  
double weight;  
double bmi;
```

If you declare a variable more than once, Java generates a compiler error.

String Concatenation

Combining several strings into a single string, or combining a string with other data into a new, longer string.

Example:

Arbitrarily complex expressions:

```
System.out.println((" + x + " " + y + " " + z + " "));
```

Increment/Decrement Operators

```
y = x + 1
```

```
y = y - 1;
```

```
z = z + 2;
```

```
x = x * 2;
```

```
y = y * 3;
```

```
z = z / 2;
```

```
x += 1;
```

```
y -= 1;
```

```
x++;
```

```
--y;
```

Description	Operators
unary operators	++, --, +, -
multiplicative operators	*, /, %
additive operators	+, -
assignment operators	=, +=, -=, *=, /=, %=

Illegal Variables

You can't mix a variable or else they become illegal example.

```
int x;
```

```
double y;
```

```
x = 3.4 + 2.9 // illegal
```

```
y = 2 + 3 // okay
```

To turn that variable into a legal variable this should be done:

```
x = (int) (3.4 * 2.9); // now legal
```

2.3 The For Loop

For loop helps avoid redundancy by repeatedly executing a sequence of statements. Instead of writing a lot of different numbers that have the same statement, you can write down a loop.

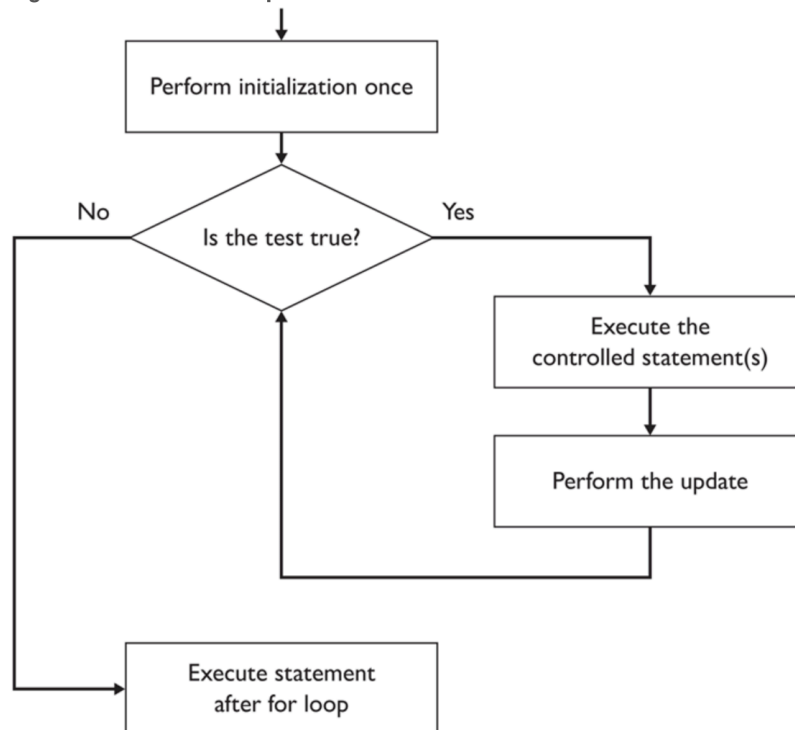
Example:

```
System.out.println(1 + " squared = " + (1 * 1));  
System.out.println(2 + " squared = " + (2 * 2));  
System.out.println(3 + " squared = " + (3 * 3));  
System.out.println(4 + " squared = " + (4 * 4));  
System.out.println(5 + " squared = " + (5 * 5));
```

You can write it as:

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i + " squared = " + (i * i));  
}
```

Figure 2.1 Flow of for loop



Other examples:

```
For (int i = 1; i <= 10; i++) {  
    System.out.print(i + " ");  
}
```

Output:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Nested Loops:

The for loop controls a statement, and the for loop is itself a statement, which means that one for loop can control another for a loop.

Example:

```
for (int i = 1; i <= 10; i++) {  
    for (int j = 1; j <= 5; j++) {  
        System.out.println("Hi there");  
    }  
}
```

Which is the same as a for loop:

```
for (int i = 1; i <= 50; i++) {  
    System.out.println("Hi there");  
}
```

Print: prints on the current line of output without going into a new line

This is interesting since you can use a nested loop with the print command to do something interesting. Example. Writing 1,2,3 six times on the current line.

Example:

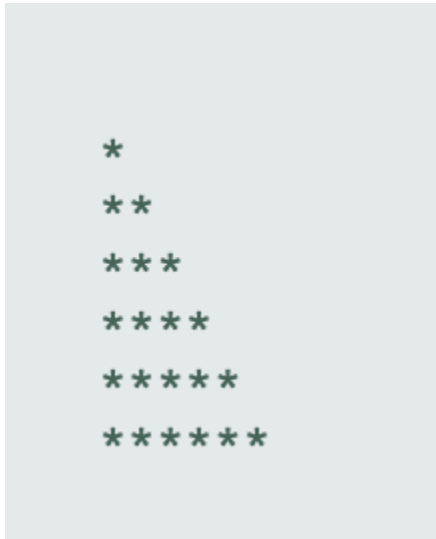
```
for (int i = 1; i <= 6; i++) {  
    for (int j = 1; j <= 3; j++) {  
        System.out.print(j + " ");  
    }  
}
```

Another way of using nested loops is increasing the number of outputs in each different line

Example:

```
for (int i = 1; i <= 6; i++) {  
    for (int j = 1; j <= i; j++) {  
        System.out.print("*");  
    }  
    System.out.println();  
}
```

Output:



```
*  
**  
***  
****  
*****  
*****
```

2.4 Managing Complexity

Scope:

The part of the program where a particular declaration is valid

Local Variable:

A variable declared inside a method that is only accessible in that method

Infinite Loop:

Loop that never terminates

Example:

```
int i;
for (int i = 1; i <= 10; i++) {
    for (int i = 1; i <= 5; i++) {
        System.out.println("Hi there");
    }
}
```

This loop is infinite since no matter what the outer loop does to the variable `i`, **the inner loop sets it back to 1, and iterates becomes 6**. The **outer loop finds the variable less than 10**, so it goes back into the inner loop, **making this process always happen**.

Referring to the Wrong Loop Variable:

(the following code is intended to print a triangle of stars, however it has a bug that causes it to print stars infinitely:

```
for (int i = 1; i <= 6; i++) {
    for (int j = 1; j <= 6; i++) {
        System.out.print("*");
    }
    System.out.println();
}
```

The problem is in the second line, in the inner for loop header's update statement. The programmer meant to write `j++` but accidentally wrote `i++`.

Step	Code	Description
initialization	<code>int i = 1;</code>	variable <code>i</code> is created and initialized to <code>1</code>
initialization	<code>int j = 1;</code>	variable <code>j</code> is created and initialized to <code>1</code>
test	<code>j <= i</code>	true because <code>1 <= 1</code> , so we enter the inner loop
body	<code>{...}</code>	execute the <code>print</code> with <code>j</code> equal to <code>1</code>
update	<code>i++</code>	increment <code>i</code> , which becomes <code>2</code>
test	<code>j <= i</code>	true because <code>1 <= 2</code> , so we enter the inner loop
body	<code>{...}</code>	execute the <code>print</code> with <code>j</code> equal to <code>1</code>
update	<code>i++</code>	increment <code>i</code> , which becomes <code>3</code>
...	...	...

Here is another piece of broken code:

(This one tries to print a 6x4 box of stars but prints infinitely instead)

```

for (int i = 1; i <= 6; i++) {
    for (int j = 1; i <= 4; j++) {
        System.out.print("*");
    }
    System.out.println();
}

```

The problem is on the second line where `i <= 4` is assigned instead of `j <= 4`. Making the code an infinite loop.

Pseudocodes:

English-like descriptions of algorithms. Programming with pseudocodes involves successively refining an informal description until it's easily translated into Java.

Example:

Draw a box with 50 lines and 30 columns of asterisks.

```
for (each of 50 lines) {  
    draw a line of 30 asterisks.  
}
```

```
for (each of 50 lines) {  
    for (each of 30 columns) {  
        write one asterisk on the output line.  
    }  
    go to a new output line.  
}
```

Using pseudocode you can convert english description into something that is easily translated into a java program.

So now we will examine the problem of generating a more complex figure:

```
*****  
*****  
*****  
***  
*
```

```
for (each of 5 lines) {  
    draw one line of the triangle.  
}
```

Since each line is different, you must come up with a general rule that fits all the lines. Using your imagination you can write something like “First line has 0 spaces, on it followed by a series of asterisks.


```

for (each of 5 lines) {
    write some spaces (possibly 0) on the output line.
    write some asterisks on the output line.
    go to a new output line.
}

```

In order to proceed **you must determine a rule for the number of spaces and a rule for the number of asterisks**. You want to find a relationship between line number and these other two values. This is simple algebra, because the values are related in a linear way.

Table 2.7 Character Counts

line	value1	value2	value3	value4
1	6	3	10	4
2	10	6	8	3
3	14	9	6	2
4	18	12	4	1
5	22	15	2	0

Table 2.8 Analysis of Figure

Line	Spaces	Asterisks
1	0	9
2	1	7
3	2	5
4	3	3
5	4	1

Using these expressions, we can improve our pseudocode as follows:

```

for (line going 1 to 5) {
    write (line - 1) spaces on the output line.
    write (11 - 2 * line) asterisks on the input line.
    go to a new output line.
}

```

This pseudocode is simple to turn into a program:

```
public class DrawV {  
    public static void main(String[] args) {  
        for (int line = 1; line <= 5; line++) {  
            for (int i = 1; i <= (line - 1); i++) {  
                System.out.print(" ");  
                {  
                    for (int i = 1; i <= (11 - 2 * line); i++) {  
                        System.out.print("*");  
                        {  
                            System.out.println();  
                        }  
                    }  
                }  
            }  
        }  
    }  
}
```

you could use the same process to make the figure **invert**, by inverting the **<= into a >=** and **turning the line++s into line--s**

